SOFTWARE REQUIREMENTS SPECIFICATION

VenueVista: EventPro Ticketing & Venue Management

Event Ticketing and Venue Management with Seating Charts,
Ticket Sales and Sales Analytics

Item	Details
Domain	Media and Entertainment
Project Type	Full Stack Development
Frontend	Angular
Backend	Java + Spring Boot
Database	Oracle Database
Core Modules	User, Event, Venue, Seating, Ticket, Booking, Sales, Analytics
AI Feature	AI-Based Personalized Event Recommendation Engine

1. Introduction & Purpose

VenueVista is a centralized full-stack web application for event, venue, seating, ticket booking, ticket sales and sales analytics. The system is intended to replace disconnected/manual operations with a structured workflow from event creation and venue allocation to seat selection, booking and sales analysis.

1.1 Scope

In Scope	Out of Scope
Users, events, venues, seats	Physical ticket printing
Ticket categories/prices	Advanced external platform integration
Booking and sales records	Biometric authentication
Sales analytics	Large-scale distributed deployment
AI event recommendations	Advanced predictive forecasting

1.2 Intended Users

- Customers — browse events, select seats and book tickets.
- Administrators — manage events, venues, seating and ticket categories.
- Management — view sales, revenue and event performance.

2. System Architecture & Components

VenueVista High-Level System Architecture

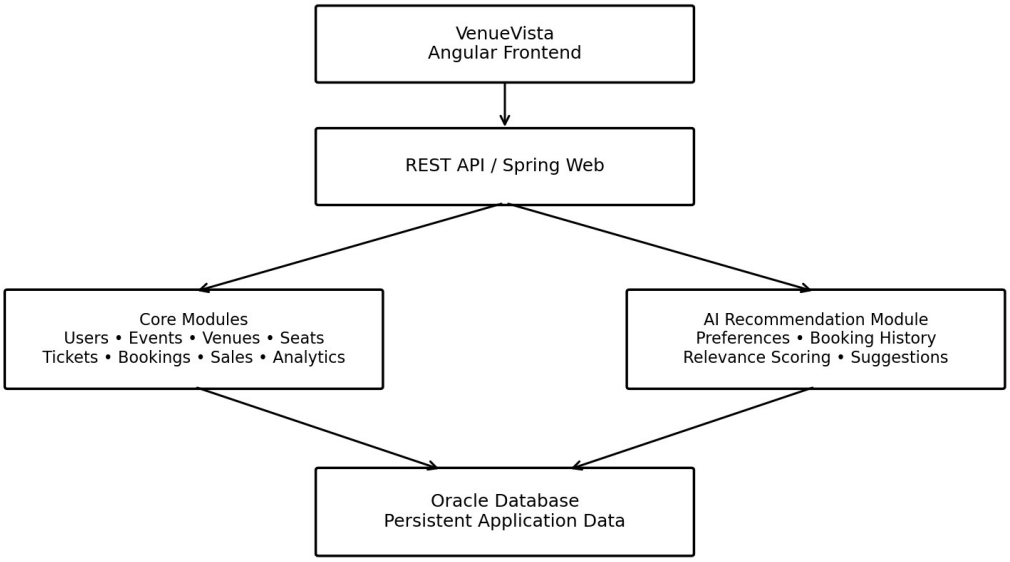


Figure 1. High-level VenueVista architecture.

2.1 Major Components

Component	Responsibility
User Management	Manage user accounts and information.
Event / Venue	Create events; maintain venue details, capacity and availability.
Seating	Configure layouts and seat availability.
Ticket / Booking	Manage ticket types, prices, seats and customer bookings.
Sales / Analytics	Record sales and provide revenue and performance information.
AI Recommendation Engine	Use preferences, booking history and event characteristics to suggest relevant events.

2.2 Technology Stack

Layer	Technology	Purpose
Frontend	Angular / TypeScript	Interactive web interface
Backend	Java / Spring Boot	Business logic and REST APIs
Persistence	Spring Data JPA / Hibernate	Database operations and ORM
Database	Oracle Database	Persistent storage
Tools	Maven, Eclipse, Postman, Git	Build, development, API testing and version control

2.3 Application Flow

User → Angular Frontend → REST API → Spring Boot Controller → Service Layer → Repository/JPA → Oracle Database → Response to Angular.

3. Data Requirements & ER Design

VenueVista Logical ER Diagram

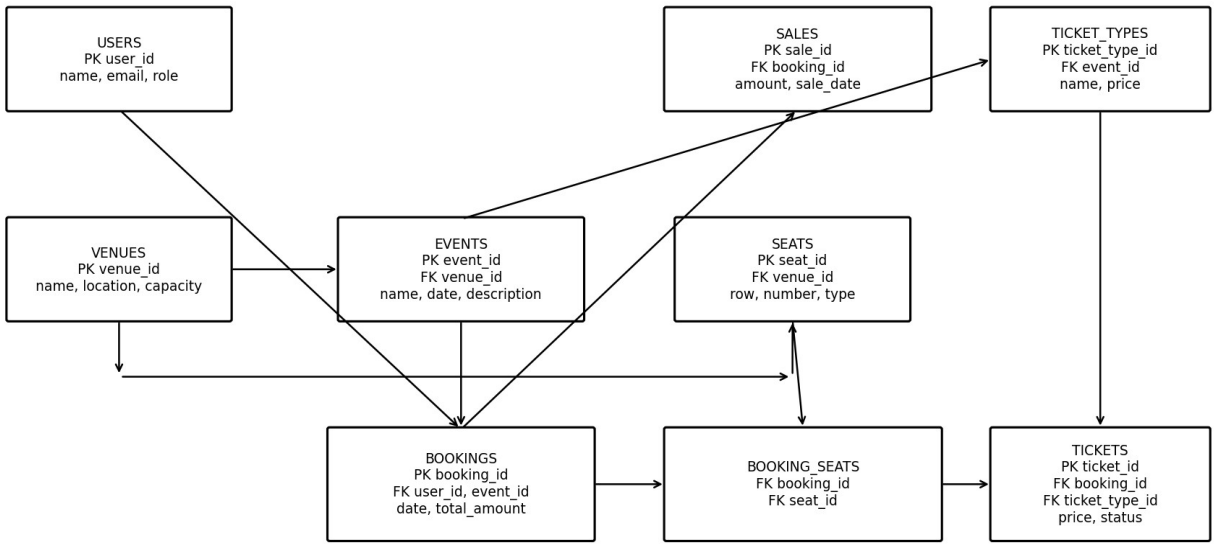


Figure 2. Logical ER diagram for core and AI-related data.

3.1 Major Entities

Entity	Key Information
Users	user_id, name, email, password, role
Venues	venue_id, name, location, capacity
Events	event_id, venue_id, name, date, description
Seats	seat_id, venue_id, seat number/type, availability
Ticket Types	ticket_type_id, event_id, type name, price
Bookings	booking_id, user_id, event_id, date, total amount
Booking Seats	booking_id, seat_id
Tickets	ticket_id, booking_id, ticket_type_id, price, status
Sales	sale_id, booking_id, amount, sale_date
User Preferences	preference_id, user_id, category, location, price range

3.2 Database Principles

- Events are associated with venues; seats are associated with venues.
- Ticket types belong to events; bookings belong to users and events.
- Booking Seats links bookings with selected seats.
- Tickets link bookings with ticket types; sales link completed bookings with sale records.
- Primary and foreign keys maintain data integrity and reduce unnecessary duplication.

4. Functional Requirements, Workflow & APIs

4.1 Core Functional Requirements

- Administrators shall create, view, update and delete events and venues.
- Administrators shall configure seating, ticket categories, prices and availability.
- Users shall browse events, view available seats and select ticket categories.
- The system shall validate seat availability, calculate booking amounts and store bookings.
- The system shall record tickets and sales and provide sales/revenue analytics.

4.2 Booking Workflow

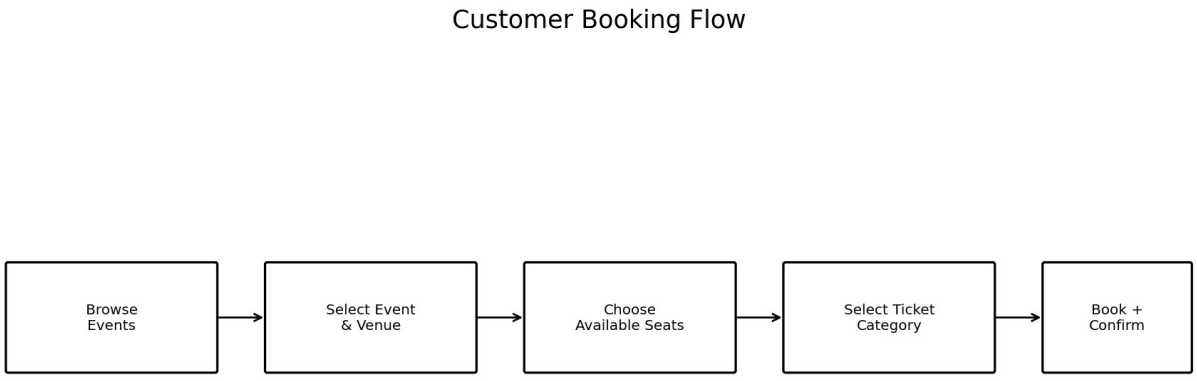


Figure 3. Customer booking workflow.

4.3 REST API Requirements

Method	Endpoint	Purpose
GET	/api/events	List events
POST	/api/events	Create event
GET	/api/events/{id}	Get event
PUT	/api/events/{id}	Update event
DELETE	/api/events/{id}	Delete event
GET / POST	/api/venues	List/create venues
GET	/api/events/{id}/seats	Get seating
POST	/api/bookings	Create booking
GET	/api/bookings/{id}	Get booking
DELETE	/api/bookings/{id}	Cancel booking
GET	/api/tickets/{id}	Get ticket
GET	/api/analytics/sales	Sales analytics
GET	/api/recommendations/{userId}	Recommendations

5. AI, Non-Functional Requirements & Acceptance

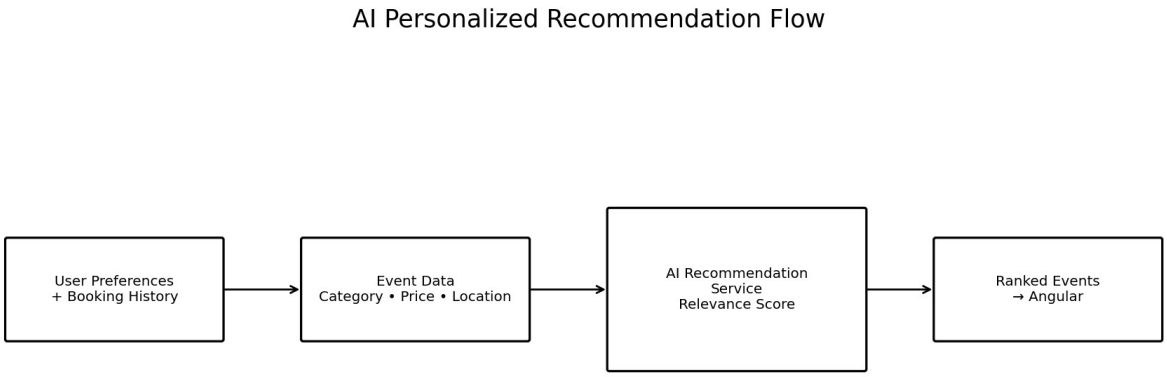


Figure 4. AI personalized event recommendation flow.

5.1 AI Recommendation Process

1. Collect user preferences and previous booking/interaction information.
2. Retrieve upcoming events and relevant characteristics.
3. Compare category, location, price range, availability and event characteristics.
4. Calculate relevance, rank suitable events and return the highest-relevance results.
5. Display personalized recommendations through the Angular frontend.

5.2 Non-Functional Requirements

Area	Requirement
Performance	Timely responses for common browsing, booking and administration operations.
Scalability	Support additional users, events, venues, bookings and ticket records.
Security	Protect credentials and sensitive information; do not expose database credentials.
Reliability	Persist booking and ticket information.
Maintainability	Keep Controller, Service, Repository and Entity layers separated.
Usability	Provide an intuitive interface for browsing, seat selection and booking.
Data Integrity	Maintain valid relationships using appropriate database constraints.

5.3 Acceptance Criteria

- Angular communicates with Spring Boot through REST APIs.
- Events and venues can be created, viewed, updated and deleted.
- Seats, ticket categories and bookings can be managed.
- Sales and revenue information can be displayed.
- AI recommendations can be generated from supported preference and event data.

6. Conclusion

VenueVista defines a centralized event ticketing and venue-management workflow for the media and entertainment domain. The modular architecture supports event, venue, seating, ticket, booking, sales and analytics operations while the AI recommendation engine adds personalized event discovery.